

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Sanchit Kashyap (1BM23CS298)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug 2025 to Dec 2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Sanchit Kashyap(1BM23CS298)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Dr. Seema Patil Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

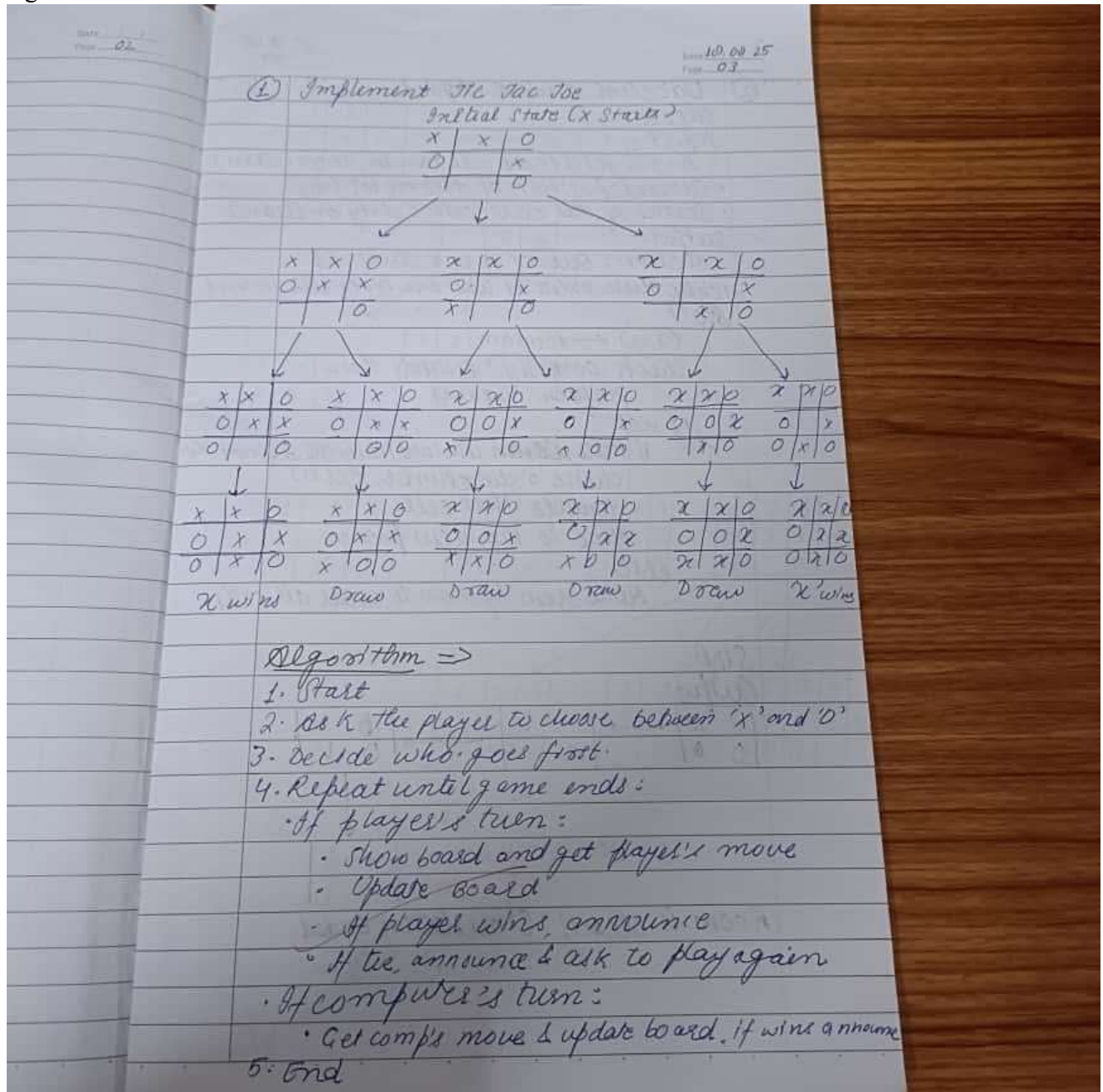
Sl. No.	Date	Experiment Title	Page No.
1	18-08-2025	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	4-9
2	25-08-2025	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	10-19
3	25-08-2025	Implement A* search algorithm	20-25
4	15-09-2025	Implement Hill Climbing search algorithm to solve N-Queens problem	26-29
5	15-09-2025	Simulated Annealing to Solve 8-Queens problem	30-32
6	22-09-2025	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	33-37
7	13-10-2025	Implement unification in first order logic	38-43
8	13-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	44-47
9	27-10-2025	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	48-51
10	16-11-2025	Implement Alpha-Beta Pruning.	52-55

Github link- <https://github.com/sanchit901/AI-1BM23CS298>

AI LAB				
Sl no.	Date	Title	Page no.	Y each is Remarks
1	18/8/25	Tic Tac Toe	10	Done 15/8/25
2	25/8/25	Vacuum cleaner	10	Done 15/8/25
3	25/8/25	(3a) BFS without Heuristic	8 puzzle	D 8-7
		(3b) BFS with II		
		3(c) DFS		
(4)	11/9/25 28/8/25	(3d) Iterative deepening		
(5)	8/9/25	Misplaced Tiles Manhattan Dist ⁿ	10	D 8-7
(6)	15/9/25	Hill climbing X Simulated Annealing	10	D
(7)	22/9/25	Proportion analog	80	D 13-10
(8)	13/10/25	Unification in FOL	10	D
(9)	13/10/25	Forward Reasoning	8	D 11-10

Implement Tic – Tac – Toe Game
Implement vacuum cleaner agent

Algorithm:



② Vacuum cleaner Agent

Algorithm:-

Input:

- A 2D grid (each cell can be Dirty or Clean)
- Current position of the robot (x, y)
- Status of the curr cell (Dirty or Clean)

Output:

- Action: One of {L, R, D, U, S}

While there exists at least one Dirty cell in Grid
Start

(x, y) ← Position

Check grid[x][y] is Dirty then
perform cleaning

if not

If there is an unvisited / dirty cell adjacent then
choose a direction (L, R, U, D)
towards that cell

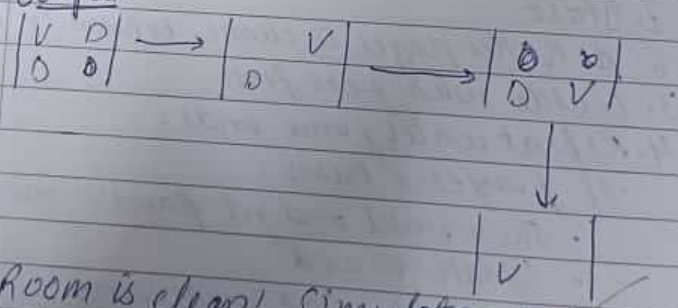
Move to the new position

else

Move row by row to cover all cells

Stop

Output:



Room is clean! Simulation ended

Code:

```
Tic tac toe: def print_board(board):
    for row in board:
        print(" ".join(row))
    print()

def check_winner(board, player):
    for i in range(3):
        if all(board[i][j] == player for j in range(3)):
            return True
    for j in range(3):
        if all(board[j][i] == player for j in range(3)):
            return True
    for i in range(3):
        if all(board[i][i] == player for i in range(3)):
            return True
    for i in range(3):
        if all(board[i][2 - i] == player for i in range(3)):
            return True
    return False

def is_draw(board):
    return all(board[i][j] != '-' for i in range(3) for j in range(3))

def minimax(board, is_ai_turn):
    if check_winner(board, 'O'):
        return 1
    if check_winner(board, 'X'):
        return -1
    if is_draw(board):
        return 0

    if is_ai_turn:
        best_score = -float('inf')
        for i in range(3):
            for j in range(3):
                if board[i][j] == '-':
                    board[i][j] = 'O'
                    score = minimax(board, False)
                    board[i][j] = '-'
                    best_score = max(score, best_score)
        return best_score
    else:
        best_score = float('inf')
        for i in range(3):
            for j in range(3):
                if board[i][j] == '-':
                    board[i][j] = 'X'
                    score = minimax(board, True)
                    board[i][j] = '-'
                    best_score = min(score, best_score)
        return best_score
```

```

board[i][j] == '-':
board[i][j] = 'X'
score = minimax(board,
True)
board[i][j] = '-'
best_score = min(score,
best_score)    return
best_score

def manual_game():
    board = [['-' for _ in range(3)] for _ in range(3)]
    print("Initial Board:")
    print_board(board)

    while True:
    while True:
    try:
        x_row = int(input("Enter X row (1-3): ")) - 1
    x_col = int(input("Enter X col (1-3): ")) - 1
    if board[x_row][x_col] == '-':
    board[x_row][x_col] = 'X'
        break    else:
    print("Cell occupied!")
    except:
        print("Invalid input!")

    print("Board after X move:")
    print_board(board)

    if check_winner(board, 'X'):
        print("X
wins!")    break
    if is_draw(board):
    print("Draw!")
        break

    while True:
    try:
        o_row = int(input("Enter O row (1-3): ")) - 1
    o_col = int(input("Enter O col (1-3): ")) - 1
    if board[o_row][o_col] == '-':
    board[o_row][o_col] = 'O'
        break    else:
    print("Cell occupied!")
    except:

```



```

        print("Invalid input!")

    print("Board after O move:")
    print_board(board)

    if check_winner(board, 'O'):
        print("O
wins!")
        break
    if is_draw(board):
        print("Draw!")
        break

    cost = minimax(board, True)
    print(f'AI
evaluation cost from this position: {cost}')

manual_game()
print("Name: Sanchit Kashyap and USN : 1BM23CS298")

```

```

vacuum cleaner:
rooms = int(input("Enter Number of rooms: "))
Rooms = "ABCDEFGHJKLMNOPQRSTUVWXYZ"
cost = 0
Roomval = {}

```

```

for i in range(rooms):
    print(f'Enter Room {Rooms[i]} state (0 for clean, 1 for dirty): ')
    n = int(input())
    Roomval[Rooms[i]] = n

```

```

loc = input(f'Enter Location of vacuum ({Rooms[:rooms]}): ').upper()

```

```

while 1 in Roomval.values():
    if Roomval[loc] == 1:
        print(f'Room
{loc} is dirty. Cleaning...')
        Roomval[loc] = 0
    cost += 1
    else:
        print(f'Room {loc} is already clean.')

```

```

move = input("Enter L or R to move left or right (or Q to quit): ").upper()

```

```

    if move == "L":
    if loc != Rooms[0]:

```

```

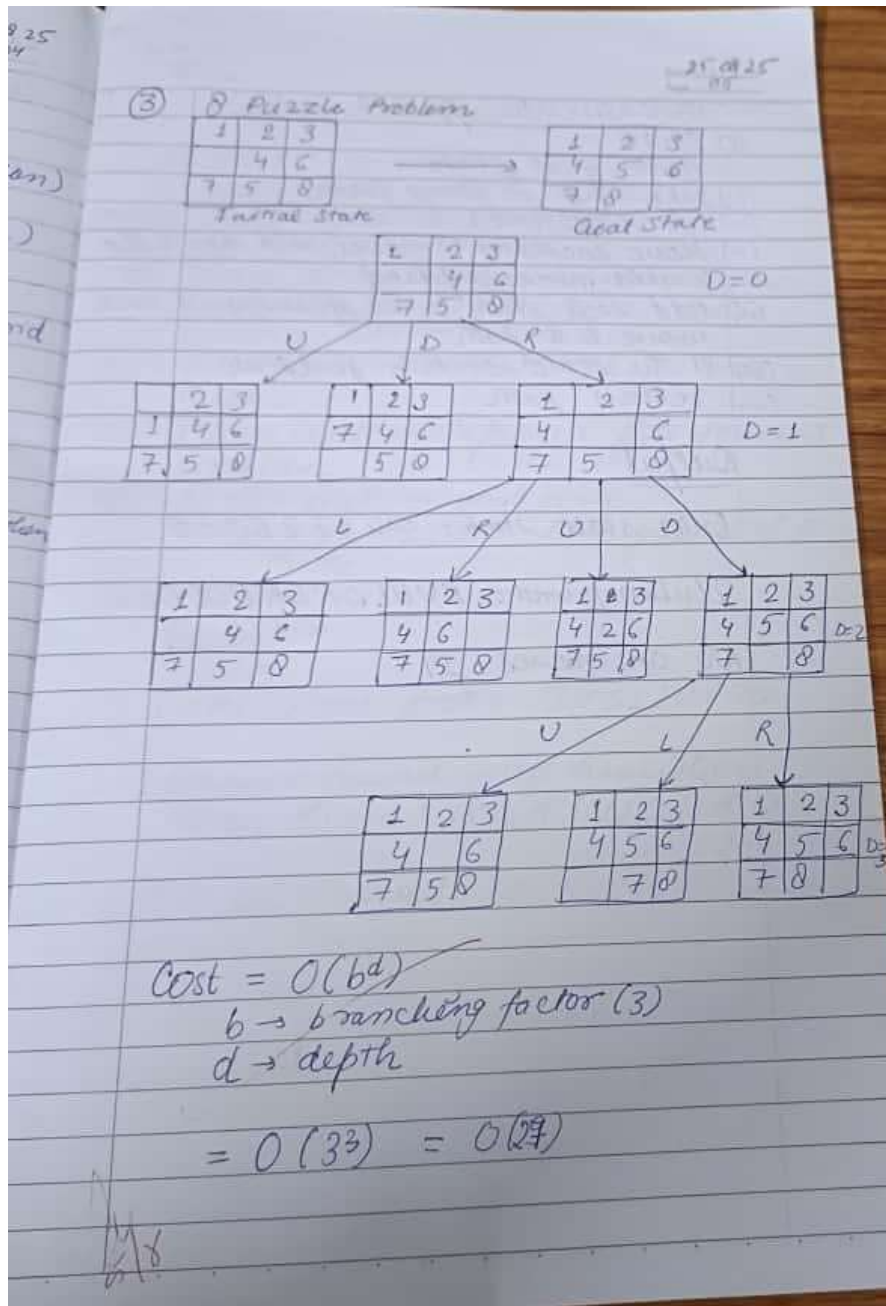
        loc = Rooms[Rooms.index(loc) - 1]
    else:
        print("No room to move left.")
    elif move == "R":
        if loc != Rooms[rooms - 1]:
            loc = Rooms[Rooms.index(loc) + 1]
        else:
            print("No room to move right.")
    elif move == "Q":
        break
    else:
        print("Invalid input. Please enter L, R, or Q.")

print("\nAll Rooms Cleaned." if 1 not in Roomval.values() else "Exited before cleaning all rooms.")
print(f'Total cost: {cost}') print("1BM23CS298")
print("Sanchit Kashyap")

```

Program 2

Implement 8 puzzle problems using Depth First Search (DFS):



Non-heuristic approach in BFS

- (i) Start
- (ii) Write goal state
- (iii) Take input in string form
- (iv) bfs (start state)
- (v) Move according to moves and don't do invalid moves updated
- (vi) Add each state to a queue and each move to a path
- (vii) If the state state = goalstate
- (viii) return path

Output

Enter start state: 345126078

Solution found: RUULORDRDUULDRD

No. of moves: 14

Heuristic in BFS

- (i) Start
- (ii) Write Goal state
- (iii) Take input in string form
- (iv) bfs (startState)
- (v) Move according to valid moves & choose least misplaced tile for next bfs

(vi) Push each h.v, path chosen into a queue

(vii) Choose least misplaced tile for next iteration

(viii) If the state = goal state
→ display results else goto step 5.

Output:-

Enter start state: 345120678

Solution found using heuristic:

Mov: RUULDRDRUULDRD

No. of moves: 14

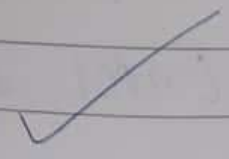
→ DFS

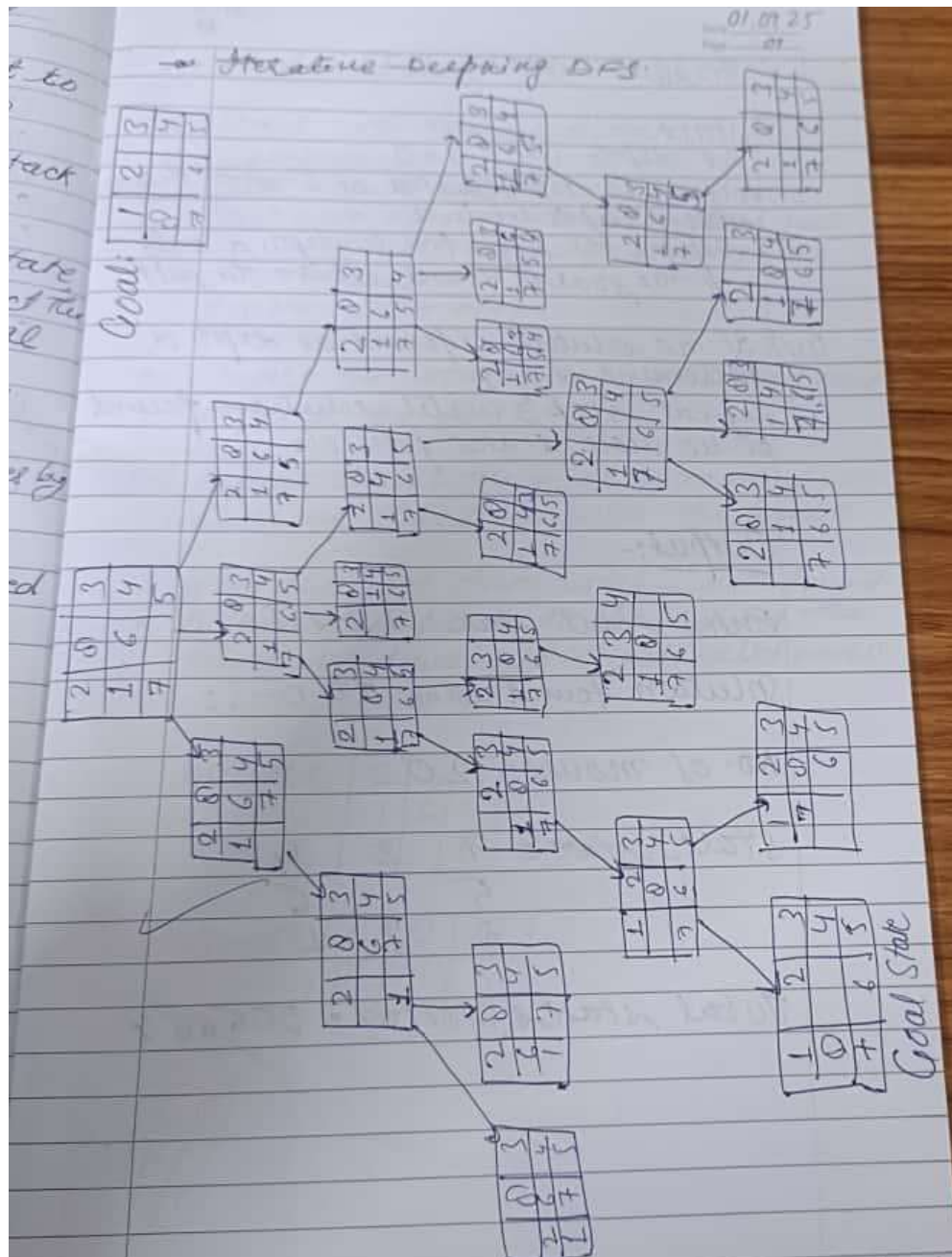
- Initialize a stack for DFS & a set to keep track of visited states to prevent cycles.
- Push the initial state onto the stack.
- While the stack is not empty:
 - Pop the curr. state from stack.
 - If the curr. state is the goal state a solution is found. Deconstruct the path from initial state to the goal state.
 - Add the curr. state to the set of visited states.
 - Generate all valid successor states by moving the blank tile.
 - For each successor state:
 - If the successor state has not been visited push it onto the stack.

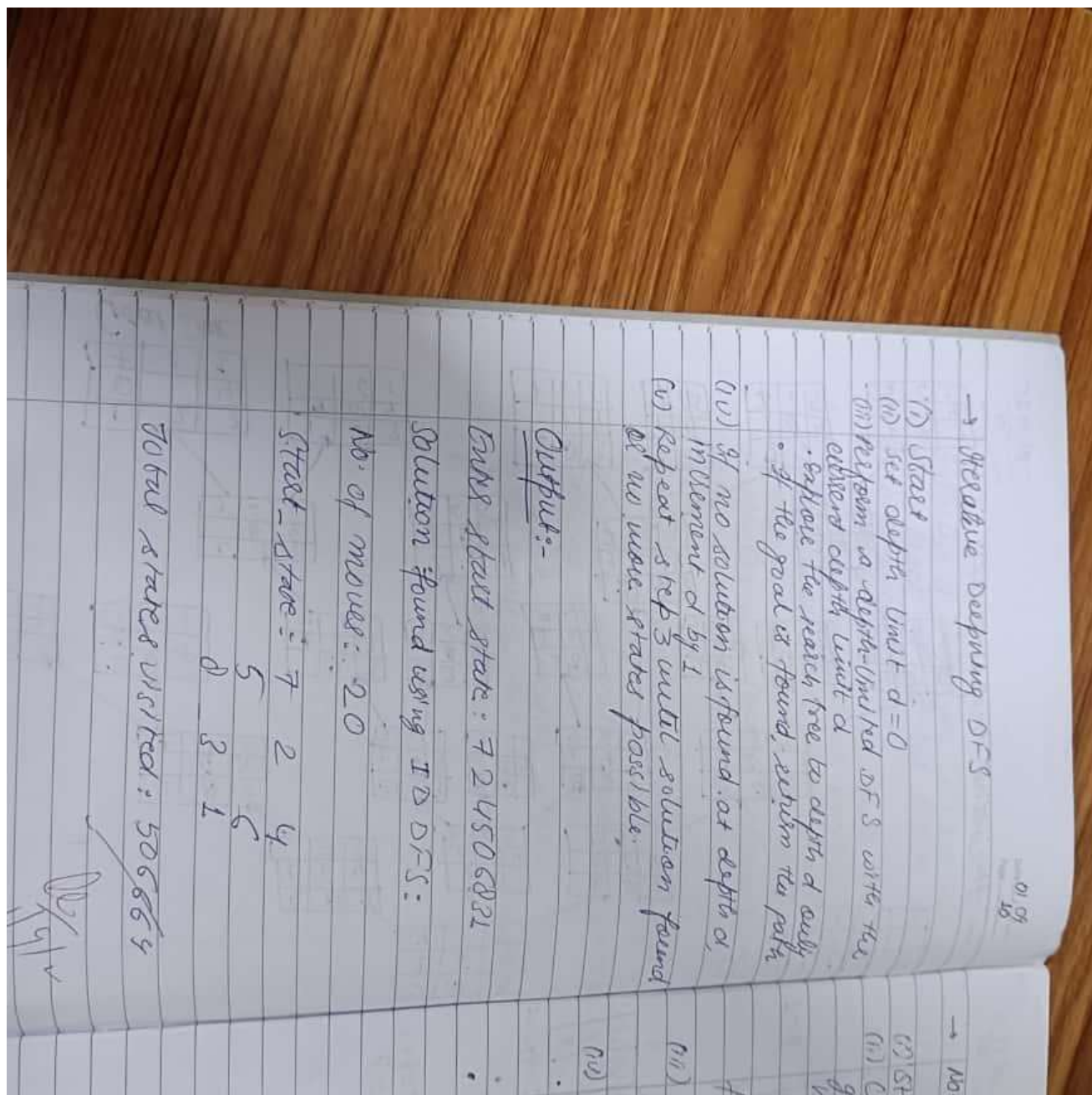
3	4
1	2

Goal:

5
6
7
8







```
goal_state = '123804765'
```

```
moves = {
    'U': -3,
    'D': 3,
    'L': -1,
    'R': 1 } count =
0 invalid_moves =
{
    0: ['U', 'L'], 1: ['U'], 2: ['U', 'R'],
```

```

3: ['L'],      5: ['R'],
6: ['D', 'L'], 7: ['D'], 8: ['D', 'R']
}

def move_tile(state, direction):
    index = state.index('0')    if direction in
invalid_moves.get(index, []):
    return None

    new_index = index + moves[direction]
if new_index < 0 or new_index >= 9:
    return None

    state_list = list(state)    state_list[index], state_list[new_index] =
state_list[new_index], state_list[index]    return ".join(state_list)

def print_state(state):
for i in range(0, 9, 3):
    print(' '.join(state[i:i+3]).replace('0', ' '))
print()

def dfs(start_state, max_depth=50):
    visited = set()    stack = [(start_state, [])] # Each
element: (state, path)

    while stack:
        current_state, path = stack.pop()

        if current_state in visited:
            continue

        # Print every visited state
print("Visited state:")
        print_state(current_state)

        if current_state == goal_state:
            return path

        visited.add(current_state)
global count    count +=1
if len(path) >= max_depth:
    continue

    for direction in moves:
        new_state = move_tile(current_state, direction)
if new_state and new_state not in visited:
    stack.append((new_state, path + [direction]))

```

```

    return None

start = input("Enter start state (e.g., 724506831): ")

if len(start) == 9 and set(start) == set('012345678'):
    print("Start state:")
    print_state(start)

    result = dfs(start)

    if result is not None:
        print("Solution found!")
        print("Moves:", ' '.join(result))
        print("Number of moves:", len(result))
        print("Number of visited states", count)
        print("IBM23CS299 Sanchit Mehta\n")

    current_state = start
    for i,
move in enumerate(result, 1):
        current_state = move_tile(current_state, move)
        print(f"Move {i}: {move}")
        print_state(current_state)
    else:
        print("No solution exists for the given start state or max depth reached.")
    else:
        print("Invalid input! Please enter a 9-digit string using digits 0-8 without repetition.")

```

Implement Iterative deepening search algorithm:

```
goal_state = '123456780'
```

```

moves = {
    'U': -3,
    'D': 3,
    'L': -1,
    'R': 1
}

invalid_moves = {
    0: ['U', 'L'], 1: ['U'], 2: ['U', 'R'],
    3: ['L'],      5: ['R'],
    6: ['D', 'L'], 7: ['D'], 8: ['D', 'R']
}

```

```

def move_tile(state, direction):
    index = state.index('0')
    if direction in invalid_moves.get(index, []):
        return None

```

```

    new_index = index + moves[direction]
if new_index < 0 or new_index >= 9:
    return None

    state_list = list(state)    state_list[index], state_list[new_index] =
state_list[new_index], state_list[index]    return ".join(state_list)

def print_state(state):
for i in range(0, 9, 3):
    print(' '.join(state[i:i+3]).replace('0', ' '))
print()

def dls(state, depth, path, visited, visited_count):
    visited_count[0] += 1 # Increment visited states count
if state == goal_state:
    return path

    if depth == 0:
return None

    visited.add(state)

    for direction in moves:
        new_state = move_tile(state, direction)
if new_state and new_state not in visited:
        result = dls(new_state, depth - 1, path + [direction], visited, visited_count)
if result is not None:
        return result

    visited.remove(state)
return None

def iddfs(start_state, max_depth=50):
    visited_count = [0] # Using list to pass by reference
for depth in range(max_depth + 1):    visited = set()
result = dls(start_state, depth, [], visited, visited_count)
if result is not None:
    return result, visited_count[0]
return None, visited_count[0]

# Main
start = input("Enter start state (e.g., 724506831): ")

```

```

if len(start) == 9 and set(start) == set('012345678'):
    print("Start state:")
    print_state(start)

    result, visited_states = iddfs(start,15)

    print(f"Total states visited: {visited_states}")

    if result is not None:
        print("Solution found!")
        print("Moves:", ' '.join(result))
        print("Number of moves:", len(result))
        print("1BM23CS298 Sanchit\n")

        current_state = start
        for i,
move in enumerate(result, 1):
            current_state = move_tile(current_state, move)
            print(f"Move {i}: {move}")
            print_state(current_state)
            else:
                print("No solution exists for the given start state or max depth reached.")
            else:
                print("Invalid input! Please enter a 9-digit string using digits 0-8 without repetition.")

```

Program 3

Implement A* search algorithm

→ No. of misplaced tiles

(i) Start with the initial state

(ii) Compute:

$g(n)$ = no. of moves made so far

$h(n)$ = no. of misplaced tiles (tiles not in their correct position, ignore the blank).

$f(n) = g(n) + h(n)$

(iii) Insert the state into a priority queue, ordered by $f(n)$.

(iv) Repeat until the goal is reached:

- Remove the state with the lowest $f(n)$
- If it is the goal state, stop & solution found.
- Otherwise, expand it by moving the blank up/down/left/right (valid moves).

Initial:

2	0	3
1	6	4
7		5

Goal:

1	2	3
0		4
7	6	5

→ Manhattan Distance

(i) Start with the initial state

(ii) Compute:

$g(n)$ = no. of moves made so far

$h(n)$ = Manhattan distance of all tiles =
sum of $|\text{row cell} - \text{row goal}| +$
 $|\text{col cell} - \text{col goal}|$ for each tile.

$f(n) = g(n) + h(n)$

(iii) Insert the state into a PQ, ordered by $f(n)$

(iv) Repeat until the goal is reached:

- Remove the state with the lowest $f(n)$
- If it is goal state, stop → soln found
- Otherwise, expand it by moving the blank up/down/left/right (valid moves).
- For each new state:
 - Compute new $g(n)$, count misplaced tiles for $h(n)$, & calc $f(n)$.
 - Add the new state to the queue if it hasn't been visited.

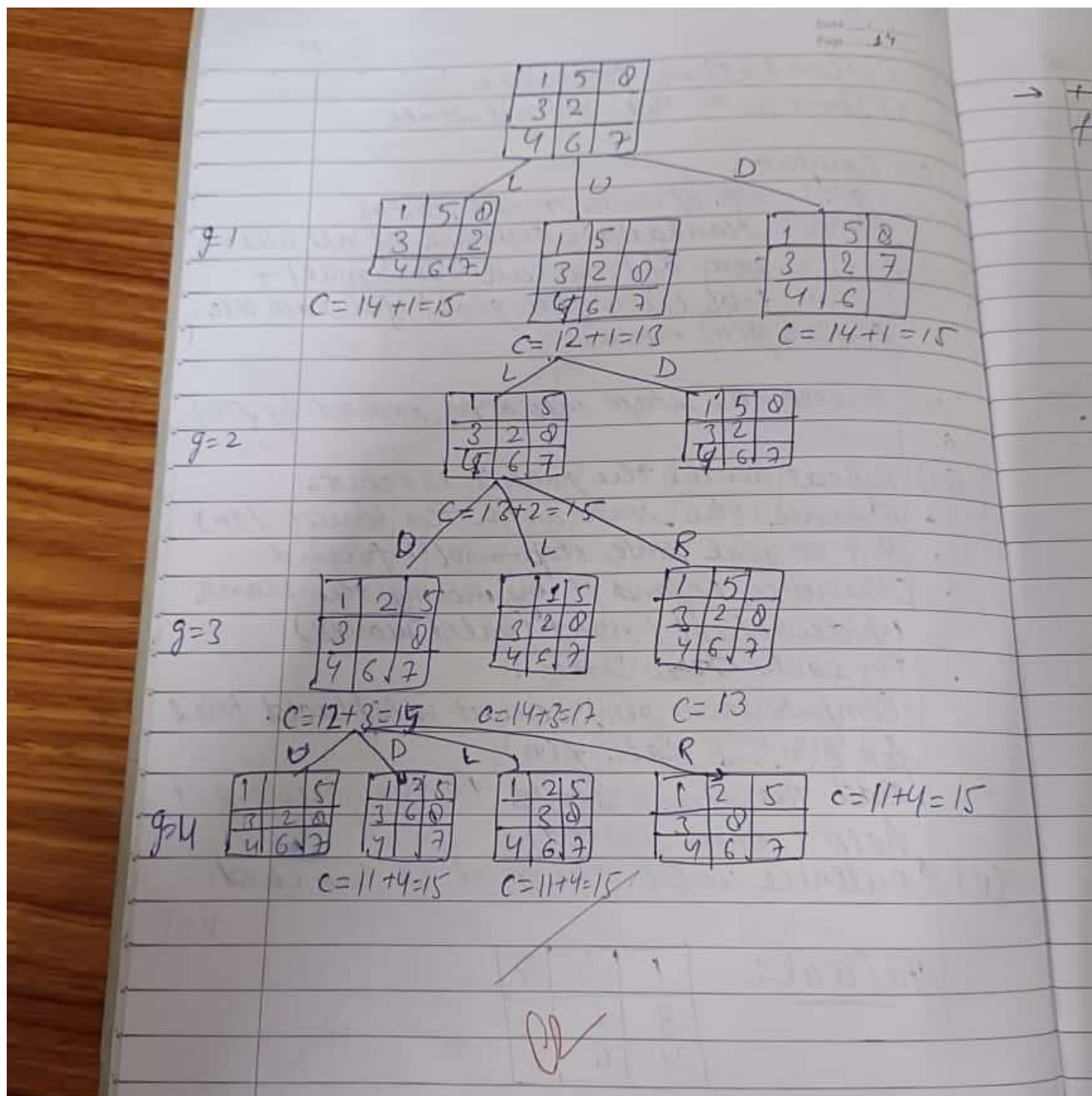
(v) Continue until the goal is reached.

Initial:

1	5	8
3	2	
4	6	7

Goal:

1	2	3
4	5	6
7	8	



```
import heapq
```

```
goal_state = '123456780'
```

```
moves = {
    'U': -3,
    'D': 3,
    'L': -1,
    'R': 1
}
```

```

}

invalid_moves = {
    0: ['U', 'L'], 1: ['U'], 2: ['U', 'R'],
    3: ['L'], 5: ['R'],
    6: ['D', 'L'], 7: ['D'], 8: ['D', 'R']
}

def move_tile(state, direction):
    index = state.index('0')    if direction in
invalid_moves.get(index, []):
        return None

    new_index = index + moves[direction]
if new_index < 0 or new_index >= 9:
    return None

    state_list = list(state)    state_list[index], state_list[new_index] =
state_list[new_index], state_list[index]    return ".join(state_list)

def print_state(state):
for i in range(0, 9, 3):
    print(' '.join(state[i:i+3]).replace('0', ' '))
print()

def manhattan_distance(state):
    distance = 0    for i, val in enumerate(state):        if val == '0':
continue        goal_pos = int(val) - 1        current_row, current_col =
divmod(i, 3)        goal_row, goal_col = divmod(goal_pos, 3)        distance
+= abs(current_row - goal_row) + abs(current_col - goal_col)    return
distance

def a_star(start_state):    visited_count = 0    open_set = []
heapq.heappush(open_set, (manhattan_distance(start_state), 0, start_state, []))
visited = set()

    while open_set:
        f, g, current_state, path = heapq.heappop(open_set)
visited_count += 1

        if current_state == goal_state:
            return path, visited_count

        if current_state in visited:
continue        visited.add(current_state)

```

```

        for direction in moves:
            new_state = move_tile(current_state, direction)
if new_state and new_state not in visited:
    new_g = g + 1
    new_f = new_g + manhattan_distance(new_state)
heapq.heappush(open_set, (new_f, new_g, new_state, path + [direction]))

return None, visited_count

# Main
start = input("Enter start state (e.g., 724506831): ")

if len(start) == 9 and set(start) == set('012345678'):
    print("Start state:")
    print_state(start)

    result, visited_states = a_star(start)

    print(f"Total states visited: {visited_states}")

    if result is not None:
        print("Solution found!")
        print("Moves:", ' '.join(result))
        print("Number of moves:", len(result))
        print("1BM23CS298 Sanchit\n")

        current_state = start
        for i,
move in enumerate(result, 1):
            current_state = move_tile(current_state, move)
        print(f"Move {i}: {move}")
    print_state(current_state) else:
        print("No
solution exists for the given start state.") else:
        print("Invalid input! Please enter a 9-digit string using digits 0-8 without repetition.")

```


Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

15.09
Page 15

→ Hill Climbing Search Algorithm
function HillClimbing(problem) returns a
state that is a local maximum
curr ← MakeNode (problem.INITIAL-STATE)
loop do
 neighbor ← a highest-valued successor of curr
 if neighbor.value ≤ curr.value then
 return curr.state
 curr ← neighbor

15

- $x_0=3, x_1=1, x_2=2, x_3=0$
cost = 2

			Q
	Q		
		Q	
Q			
- $x_0=1, x_1=0, x_2=3, x_3=2$
Cost = 2+1+1 = 4

	Q		
Q			
			Q
		Q	
- $x_0=1, x_1=3, x_2=0, x_3=2$

C = 0

	Q		
			Q
Q			
		Q	
- $x_0=3, x_1=2, x_2=0, x_3=1$
~~C = 2~~

			Q
		Q	
Q			
	Q		
- $x_0=2, x_1=0, x_2=3, x_3=1$

C = 0

		Q	
Q			
			Q
	Q		

→ Output:-

Enter the no. of queens (N): 4

Initial State (heuristic 4):

Q . . .
. . . Q
. . . .
. . Q .

Step 1: (h=4):

Q . . .
. . . Q
Q . . .
. . Q .

Step 2: (h=0)

. Q . .
. . . Q
Q . . .
. . Q .

Reached local min at step 2, h=0
Solution found after 1 restart

. Q . .
. . . Q
Q . . .
. . Q .

IBM23CS304 Saad Al-

15/5


```

import random
import time

def print_board(state):
    """Prints the chessboard for a given state."""
    n = len(state)
    for row in range(n):
        line = ""
        for col in range(n):
            if state[col] == row:
                line += "Q "
            else:
                line += ". "
        print(line)
    print()

def compute_heuristic(state):
    """Computes the number of attacking pairs of queens."""
    h = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                h += 1
    return h

def get_neighbors(state):
    """Generates all possible neighbors by moving one queen in its column."""
    neighbors = []
    n = len(state)
    for col in range(n):
        for row in range(n):
            if row != state[col]:
                neighbor = list(state)
                neighbor[col] = row
                neighbors.append(neighbor)
    return neighbors

def hill_climb_verbose(initial_state, step_delay=0.5):
    """Hill climbing algorithm with verbose output at each step."""
    current = initial_state
    current_h = compute_heuristic(current)
    step = 0

    print(f"Initial state (heuristic: {current_h}):")
    print_board(current)
    time.sleep(step_delay)

    while True:
        neighbors = get_neighbors(current)
        next_state = None
        next_h = current_h

```

```

        for neighbor in neighbors:
            h = compute_heuristic(neighbor)
        if h < next_h:
            next_state = neighbor
            next_h = h

        if next_h >= current_h:
            print(f'Reached local minimum at step {step}, heuristic: {current_h}')
            return current, current_h

        current = next_state
        current_h = next_h
        step += 1
        print(f'Step {step}: (heuristic: {current_h})')
        print_board(current)
        time.sleep(step_delay)

def solve_n_queens_verbose(n, max_restarts=1000):
    """Solves N-Queens problem using hill climbing with restarts and verbose output."""
    for attempt in range(max_restarts):
        print(f'\n=== Restart {attempt + 1} ===\n')
        initial_state = [random.randint(0, n - 1) for _ in range(n)]
        solution, h = hill_climb_verbose(initial_state)
        if h == 0:
            print(f' Solution found after {attempt + 1} restart(s):')
            print_board(solution)
            return solution
        else:
            print(f'No solution in this attempt (local minimum).\n')
    print("Failed to find a solution after max restarts.")
    return None

# --- Run the algorithm --- if
__name__ == "__main__":
    N = int(input("Enter the number of queens (N): "))
    solve_n_queens_verbose(N)
    print("1BM23CS298 Sanchit Kashyap ")

```

Program 5

Simulated Annealing to Solve 8-Queens problem

→ Simulated Annealing
 $curr \leftarrow$ initial state
 $T \leftarrow$ a large +ve value

while $T > 0$ do

$next \leftarrow$ a random neighbor of $curr$.

$\Delta E \leftarrow curr.cost - next.cost$

 if $\Delta E > 0$ then

$curr \leftarrow next$

 else

$curr \leftarrow next$ with prob. $p = e^{\Delta E / T}$

 end if

 decrease T

end while

return $curr$

Output:-

Enter no. of queens:- 8

Solutions found at step 623

Position format:

[3 1 7 4 6 0 2 5]

Heuristic 0

```
import random
import math
```

```
def compute_heuristic(state):
```

```
    """Number of attacking pairs."""    h = 0    n = len(state)
```

```
    for i in range(n):        for j in range(i + 1, n):            if state[i]
```

```
    == state[j] or abs(state[i] - state[j]) == abs(i - j):
```

```
        h += 1
```

```
    return h
```

```

def random_neighbor(state):
    """Returns a neighbor by randomly changing one queen's row."""
    n = len(state)    neighbor = state[:]    col = random.randint(0, n - 1)
    old_row = neighbor[col]    new_row = random.choice([r for r in
range(n) if r != old_row])    neighbor[col] = new_row
    return neighbor

def dual_simulated_annealing(n, max_iter=10000, initial_temp=100.0, cooling_rate=0.99):
    """Simulated Annealing with dual acceptance strategy."""
    current = [random.randint(0, n - 1) for _ in range(n)]
    current_h = compute_heuristic(current)    temperature =
initial_temp

    for step in range(max_iter):
        if current_h == 0:            print(f" Solution
found at step {step}")            return current

        neighbor = random_neighbor(current)
        neighbor_h = compute_heuristic(neighbor)
        delta = neighbor_h - current_h
        if delta <
0:
            current = neighbor
        current_h = neighbor_h        else:
            # Dual acceptance: standard + small chance of higher uphill move
            probability = math.exp(-delta / temperature)            if random.random()
< probability:
                current = neighbor
                current_h = neighbor_h

        temperature *= cooling_rate        if
temperature < 1e-5: # Restart if stuck
            temperature = initial_temp
            current = [random.randint(0, n - 1) for _ in range(n)]
            current_h = compute_heuristic(current)

    print(" Failed to find solution within max iterations.")    return
None

# --- Run the algorithm --- if
__name__ == "__main__":
    N = int(input("Enter number of queens (N): "))
    solution = dual_simulated_annealing(N)

    if solution:

```

```
print("Position format:")    print("[", "  
".join(str(x) for x in solution), "]" )  
print("Heuristic:", compute_heuristic(solution))  
print("1BM23CS298 Sanchit Kashyap ")
```

Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not:

- Propositional logic
 * Truth tables for connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$
false	false	true	false	false	true
false	true	true	false	true	false
true	false	false	false	true	false
true	true	false	true	true	true

Implementation of TT enumeration algorithm for deciding propositional entailment

e.g: $\alpha = A \vee B$
 $KB = (A \vee C) \wedge (B \vee \neg C)$

Checking that $KB \models \alpha$

A	B	C	$A \vee C$	$B \vee \neg C$	KB	α
false	false	false	false	true	false	false
false	false	true	true	false	false	false
false	true	false	false	true	false	true
false	true	true	true	true	true	true
true	false	false	true	true	true	true
true	false	true	true	false	false	true
true	true	false	true	true	true	true
true	true	true	true	true	true	true

function TT-ENTAILS?(KB, α) returns true or false
 inputs: KB, the knowledge base, a sentence in
 propositional logic α , the query, a
 sentence in propositional logic

symbols $\leftarrow$ a list of the proposition symbols
 in KB & α
 return TT-CHECK-ALL(KB, α , symbols, T)

function TT-CHECK-ALL(KB, α , symbols, model) returns T or F
 if EMPTY?(symbols) then
 if PL-TRUE?(KB, model) then return
 return PL-TRUE?(α , model)
 else return true

else do
 $p \leftarrow$ FIRST(symbols)
 $rest \leftarrow$ REST(symbols)
 return (TT-CHECK-ALL(KB, α , rest, model)
 $\cup$ { $p = \text{true}$ })
 and
 TT-CHECK-ALL(KB, α , rest, model, { $p = \text{false}$ })

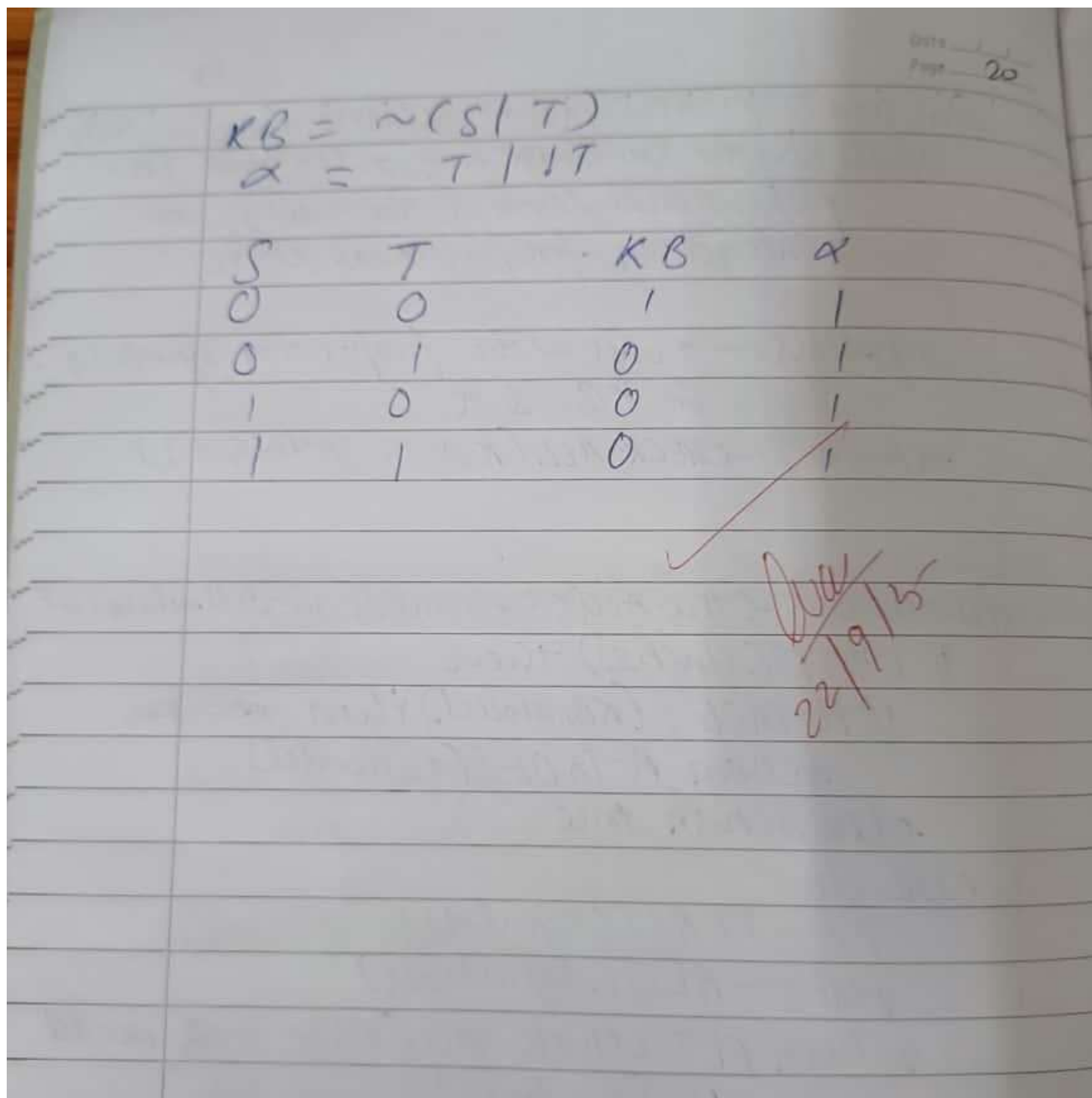
a entails b $\rightarrow$ False

EM: $KB = \sim(S \mid T)$

$\alpha = S \wedge T$

S	T	KB	α
0	0	1	0
0	1	0	0
1	0	0	0
1	1	0	1

a entails c; True



```
from typing import List, Tuple, Dict, Set, Union
```

```
Predicate = Tuple[str, Tuple[str, ...]]
```

```
class Rule:
    def __init__(self, head: Predicate, body:
List[Predicate]):
        self.head = head
        self.body = body
```

```

def __repr__(self):
    body_str = ', '.join(f'{p[0]} {p[1]}' for p in self.body)
    return f'{body_str} => {self.head[0]} {self.head[1]}'

# Knowledge base class
KnowledgeBase: def
__init__(self):
    self.facts: Set[Predicate] = set()
    self.rules: List[Rule] = []

def add_fact(self, fact: Predicate):
    self.facts.add(fact)

def add_rule(self, rule: Rule):
    self.rules.append(rule)

def forward_chain(self, query: Predicate) -> bool:
    inferred = set(self.facts)
    added = True

    while added:
        added = False
        for rule in self.rules:
            if
            all(self._match_fact(body_pred, inferred) for body_pred in rule.body):
            if not self._match_fact(rule.head, inferred):
                inferred.add(rule.head)
            added = True
            print(f'Inferred:
            {rule.head}')

        if self._match_fact(query, inferred):
            return True
        return

    self._match_fact(query, inferred)

def _match_fact(self, pred: Predicate, fact_set: Set[Predicate]) -> bool:
    return pred in fact_set

# --- Example usage ---
if __name__ == "__main__":
    kb = KnowledgeBase()

    kb.add_fact(("Parent", ("John", "Mary")))
    kb.add_fact(("Parent", ("Mary", "Sue")))

    facts_list = list(kb.facts)
    for f1 in facts_list:
        if f1[0] == "Parent"
        and f2[0] == "Parent":
            if f1[1][1] ==
            f2[1][0]:

```

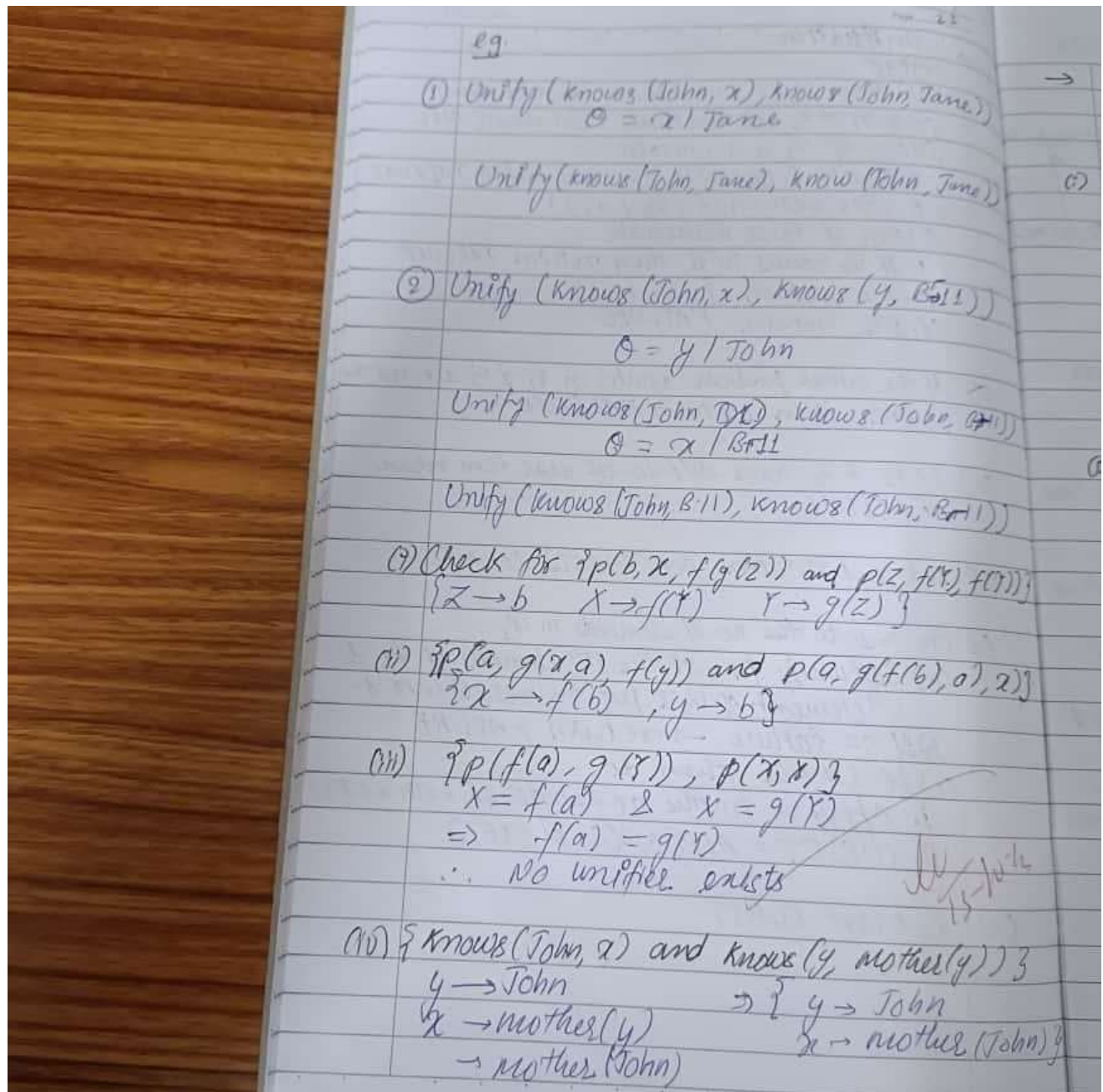
```
head = ("Grandparent", (f1[1][0], f2[1][1]))
body = [f1, f2]
kb.add_rule(Rule(head, body))

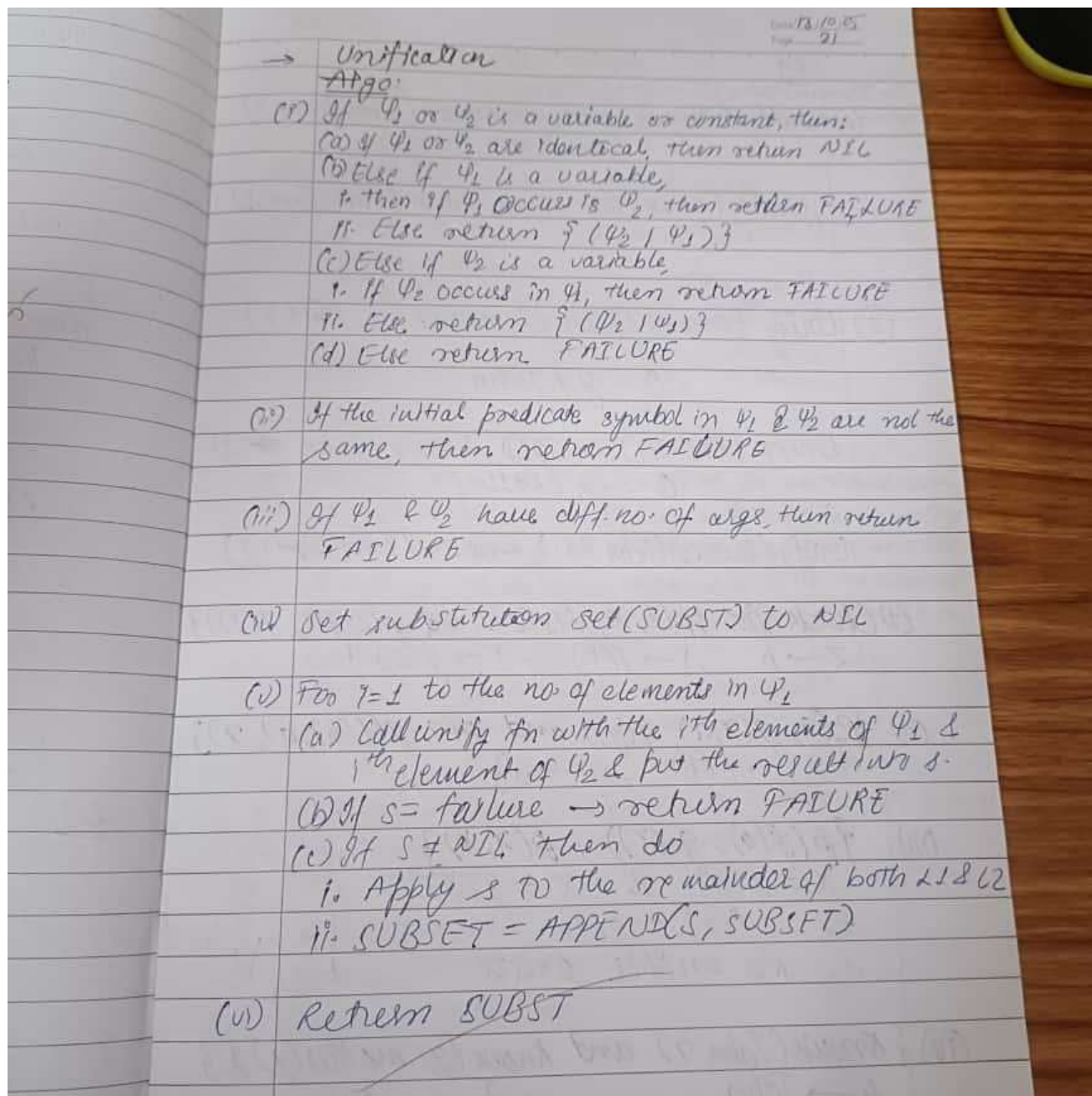
query = ("Grandparent", ("John", "Sue"))
result = kb.forward_chain(query)

print(f'Query {query} is', "True" if result else "False")
print("1BM23CS298 Sanchit Kashyap ")
```

Program 7

Implement unification in first order logic:





```
import json
```

```
# --- Helper Functions for Term Manipulation ---
```

```
def is_variable(term):
```

```
    """Checks if a term is a variable (a single capital letter string)."""
    return isinstance(term, str) and len(term) == 1 and 'A' <= term[0] <= 'Z'
```

```

def occurs_check(variable, term, sigma):
    """
    Checks if 'variable' occurs anywhere in 'term' under the current substitution 'sigma'.
    This prevents infinite recursion (e.g., unifying X with f(X)).
    """
    term = apply_substitution(term, sigma) # Check the substituted term

    if term == variable:
        return True

    # If the term is a list (function/predicate), check its arguments recursively
    if isinstance(term, list):
        for arg in term[1:]:
            if occurs_check(variable, arg, sigma):
                return True

    return False

def apply_substitution(term, sigma):
    """
    Applies the current substitution 'sigma' to a 'term' recursively.
    """
    if is_variable(term):
        # If the variable is bound in sigma, apply the binding
        if term in sigma:
            # Recursively apply the rest of the substitutions to the binding's value
            # This is critical for chains like X/f(Y), Y/a -> X/f(a)
            return apply_substitution(sigma[term], sigma)
        return term

    if isinstance(term, list):
        # Apply substitution to the arguments of the function/predicate
        new_term = [term[0]] # Keep the function/predicate symbol
        for arg in term[1:]:
            new_term.append(apply_substitution(arg, sigma))
        return new_term

    # Term is a constant or an unhandled type, return as is
    return term

def term_to_string(term):
    """
    Converts the internal list representation of a term into standard logic notation string.
    e.g., ['f', 'Y'] -> "f(Y)"
    """
    if isinstance(term, str):
        return term

```



```

    if isinstance(term, list):
        # Term is a function or predicate      symbol
    = term[0]      args = [term_to_string(arg) for arg
in term[1:]]      return f'{symbol}({' +
'.join(args)}')

    return str(term)

# --- Main Unification Function ---

def unify(term1, term2):
    """
    Implements the Unification Algorithm to find the MGU for term1 and term2.
    Returns the MGU as a dictionary or None if unification fails.
    """
    # Initialize the substitution set (MGU)
    sigma = {}

    # Initialize the list of pairs to resolve (the difference set)
    diff_set = [[term1, term2]]

    print(f"--- Unification Process Started ---")
    print(f"Initial Terms:")    print(f"L1:
{term_to_string(term1)}")    print(f"L2:
{term_to_string(term2)}")
    print("-" * 35)

    while diff_set:
        # Pop the current pair of terms to unify
    t1, t2 = diff_set.pop(0)

        # 1. Apply the current MGU to the terms before comparison
    t1_prime = apply_substitution(t1, sigma)    t2_prime =
    apply_substitution(t2, sigma)

        print(f"Attempting to unify: {term_to_string(t1_prime)} vs {term_to_string(t2_prime)}")

        # 2. Check if terms are identical
    if t1_prime == t2_prime:
        print(f" -> Identical. Current MGU: {term_to_string(sigma)}")
        continue

```

```

    # 3. Handle Variable-Term unification
    if is_variable(t1_prime):
        var, term = t1_prime, t2_prime
    elif is_variable(t2_prime):
        var, term = t2_prime, t1_prime
    else:
        var, term = None, None
    if
var:
    # Check if term is a variable, and if so, don't bind V/V
    if is_variable(term):
        print(f" -> Both are variables. Skipping {var} / {term}")
        # Ensure they are added back if not identical (which is caught by step 2).
        # If V1 != V2, we add V1/V2 or V2/V1 to sigma. Since step 2 handles V/V, this means V1
        != V2 here.
        if var != term:
            sigma[var] = term
            print(f" -> Variable binding added: {var} /
{term_to_string(term)}. New MGU:
{term_to_string(sigma)}")
            # Occurs Check: Fail if the variable occurs in the term it's being bound to
        elif occurs_check(var, term, sigma):
            print(f" -> OCCURS CHECK FAILURE: Variable {var} occurs in
{term_to_string(term)}")
            return None

        # Create a new substitution {var / term}
    else:
        sigma[var] = term
        print(f" -> Variable binding added: {var} /
{term_to_string(term)}. New MGU: {term_to_string(sigma)}")

    # 4. Handle Complex Term (Function/Predicate) unification
    elif isinstance(t1_prime, list) and isinstance(t2_prime, list):
        # Check functor/predicate symbol and arity (number of arguments)
        if t1_prime[0] != t2_prime[0] or len(t1_prime) != len(t2_prime):
            print(f" -> FUNCTOR/ARITY MISMATCH: {t1_prime[0]} != {t2_prime[0]} or arity
mismatch.")
            return None

        # Add corresponding arguments to the difference set
        # Start from index 1 (after the symbol)
        for arg1, arg2 in zip(t1_prime[1:], t2_prime[1:]):
            diff_set.append([arg1, arg2])
            print(f" -> Complex terms
matched. Adding arguments to difference set.")

    # 5. Handle Constant-Constant or other mismatches (Fail)
    else:

```

```

        print(f' -> TYPE/CONSTANT MISMATCH: {term_to_string(t1_prime)} and
{term_to_string(t2_prime)} cannot be unified.')
    return None

```

```

    print("-" * 35)    print("---
Unification Successful ---")

```

```

    # Final cleanup to ensure all bindings are fully resolved
    {k: apply_substitution(v, sigma) for k, v in sigma.items()}
    final_mgu = return
    final_mgu

```

```

# --- Define the Input Terms ---

```

```

# L1 = Q(a, g(X, a), f(Y))
literal1 = ['Q', 'a', ['g', 'X', 'a'], ['f', 'Y']]

```

```

# L2 = Q(a, g(f(b), a), X)
literal2 = ['Q', 'a', ['g', ['f', 'b'], 'a'], 'X']

```

```

# --- Run the Unification ---

```

```

mgu_result = unify(literal1, literal2)

```

```

if mgu_result is not None:
    print("\n[ Final MGU Result ]")

```

```

    # Format the final MGU for display using the new helper function
    clean_mgu = {k: term_to_string(v) for k, v in mgu_result.items()}
    final_output = ', '.join([f'{k} / {v}' for k, v in clean_mgu.items()])
    print(f'Final MGU: {{ {final_output} }}')

```

```

    # --- Verification ---    print("\n[ Verification ]")
    unified_11 = apply_substitution(literal1, mgu_result)
    unified_12 = apply_substitution(literal2, mgu_result)

```

```

    print(f'L1 after MGU: {term_to_string(unified_11)}')
    print(f'L2 after MGU: {term_to_string(unified_12)}')

```

```

    if unified_11 == unified_12:
        print("-> SUCCESS: L1 and L2 are identical after applying the MGU.")
    else:
        print("-> ERROR: Unification failed verification.")
    else:
        print("\nUnification FAILED.")

```

```

print("Sanchit Kashyap 1BM23CS298")

```

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning:

→ First Order Logic
Create a KB consisting of FOL statements & prove the given query using forward reasoning

→ Premises → Conclusion

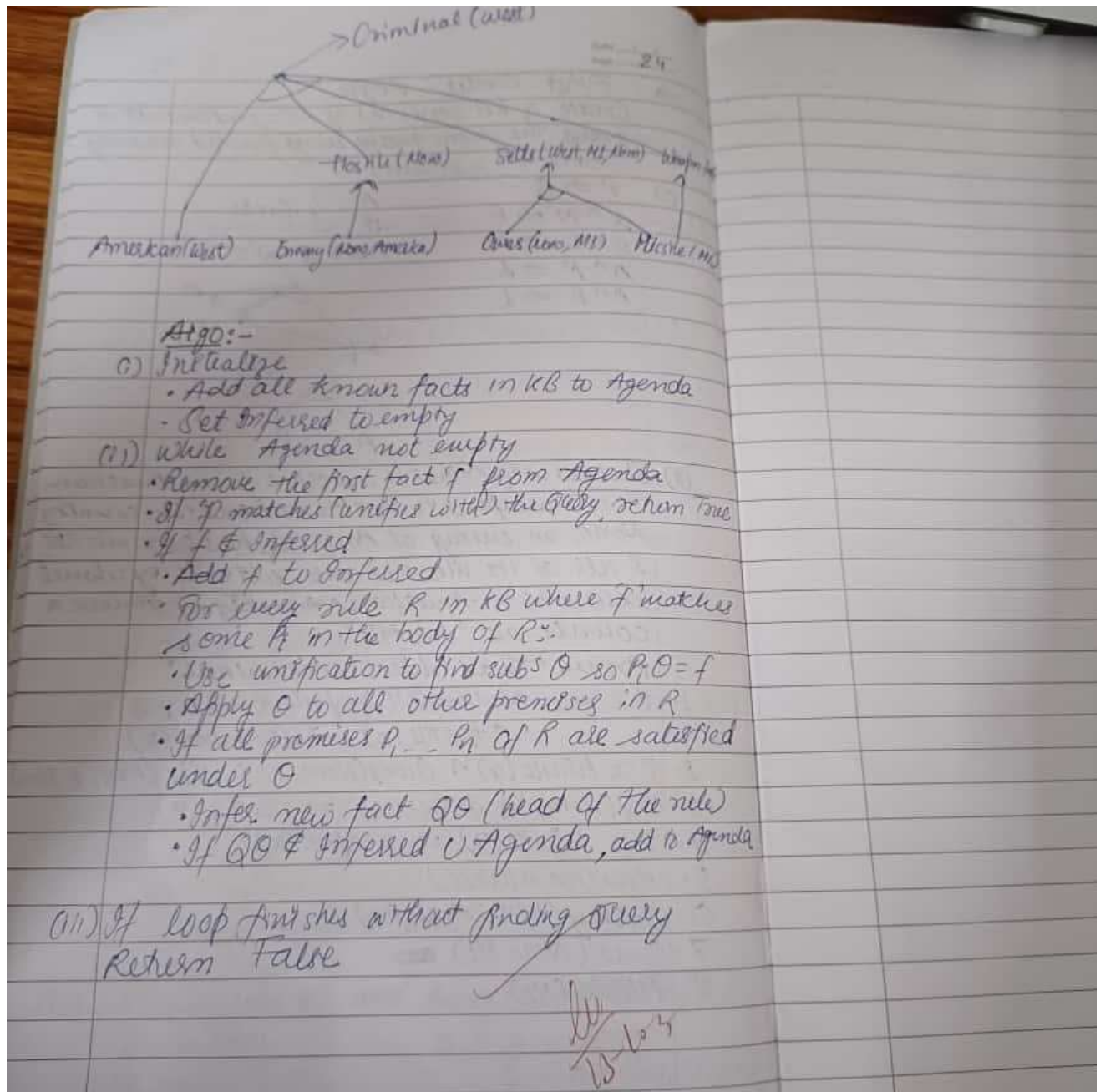
(i) $P \Rightarrow Q$

$A \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$

A, B } facts
 Prove $Q \rightarrow P \rightarrow Q$

(ii) The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles & all of its missiles were sold to it by colonel West, who is American. An enemy of America counts as "hostile".
 Prove that "West is criminal"

- $\forall x, y, z \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
- $\forall x \text{ Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
- $\forall x \text{ Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
- $\forall x \text{ Missile}(x) \Rightarrow \text{Weapon}(x)$
- $\text{American}(\text{West})$
- $\text{Enemy}(\text{Nono}, \text{America})$
- $\text{Owns}(\text{Nono}, M1)$
- $\text{Missile}(M1)$



from typing import List, Tuple, Dict, Set, Union

Predicate = Tuple[str, Tuple[str, ...]]

```
class Rule:
    def __init__(self, head: Predicate, body:
List[Predicate]):
        self.head = head
        self.body = body
```

```

def __repr__(self):
    body_str = ', '.join(f'{p[0]} {p[1]}' for p in self.body)
    return f'{body_str} => {self.head[0]} {self.head[1]}'

# Knowledge base class
KnowledgeBase: def
__init__(self):
    self.facts: Set[Predicate] = set()
    self.rules: List[Rule] = []

def add_fact(self, fact: Predicate):
    self.facts.add(fact)

def add_rule(self, rule: Rule):
    self.rules.append(rule)

def forward_chain(self, query: Predicate) -> bool:
    inferred = set(self.facts)
    added = True

    while added:
        added = False
        for rule in self.rules:
            if
            all(self._match_fact(body_pred, inferred) for body_pred in rule.body):
            if not self._match_fact(rule.head, inferred):
                inferred.add(rule.head)
            added = True
            print(f'Inferred:
            {rule.head}')

        if self._match_fact(query, inferred):
            return True
        return

    self._match_fact(query, inferred)

def _match_fact(self, pred: Predicate, fact_set: Set[Predicate]) -> bool:
    return pred in fact_set

# --- Example usage ---
if __name__ == "__main__":
    kb = KnowledgeBase()

    kb.add_fact(("Parent", ("John", "Mary")))
    kb.add_fact(("Parent", ("Mary", "Sue")))

    facts_list = list(kb.facts)
    for f1 in facts_list:
        for f2 in facts_list:
            if f1[0] == "Parent"
            and f2[0] == "Parent":
                if f1[1][1] ==
                f2[1][0]:

```

```
head = ("Grandparent", (f1[1][0], f2[1][1]))
body = [f1, f2]
kb.add_rule(Rule(head, body))

query = ("Grandparent", ("John", "Sue"))
result = kb.forward_chain(query)

print(f'Query {query} is', "True" if result else "False")
```


Create a knowledge base consisting of first order logic statements and prove the given query using Resolution.

e.g.

Given the KB or Premises:

John likes all kind of food.

Apple & vegetables are food

Anything anyone eats and not killed is food.

Bnil eats peanuts and still alive.

Harry eats everything that Bnil eats.

Someone who is alive implies not killed.

Someone who is not killed implies alive

Proof by resolution: John Likes peanuts

Solⁿ

$\forall x : \text{food}(x) \rightarrow \text{Likes}(\text{John}, x)$

$\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$

$\forall x \forall y : \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$

$\text{eats}(\text{Bnil}, \text{Peanuts}) \wedge \text{Alive}(\text{Bnil})$

$\forall x : \text{eats}(\text{Bnil}, x) \rightarrow \text{eats}(\text{Harry}, x)$

$\forall x : \neg \text{killed}(x) \rightarrow \text{alive}(x)$

$\forall x : \text{alive}(x) \rightarrow \neg \text{killed}(x)$

$\text{Likes}(\text{John}, \text{Peanuts})$

$\neg \text{Likes}(\text{John}, \text{Peanuts}) \quad \neg \text{food}(x) \vee \text{Likes}(\text{John}, x)$

$\neg \text{food}(\text{Peanuts}) \quad \neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$

$\neg \text{eats}(y, \text{Peanuts}) \vee \text{killed}(y) \quad \text{eats}(\text{Bnil}, \text{Peanuts})$

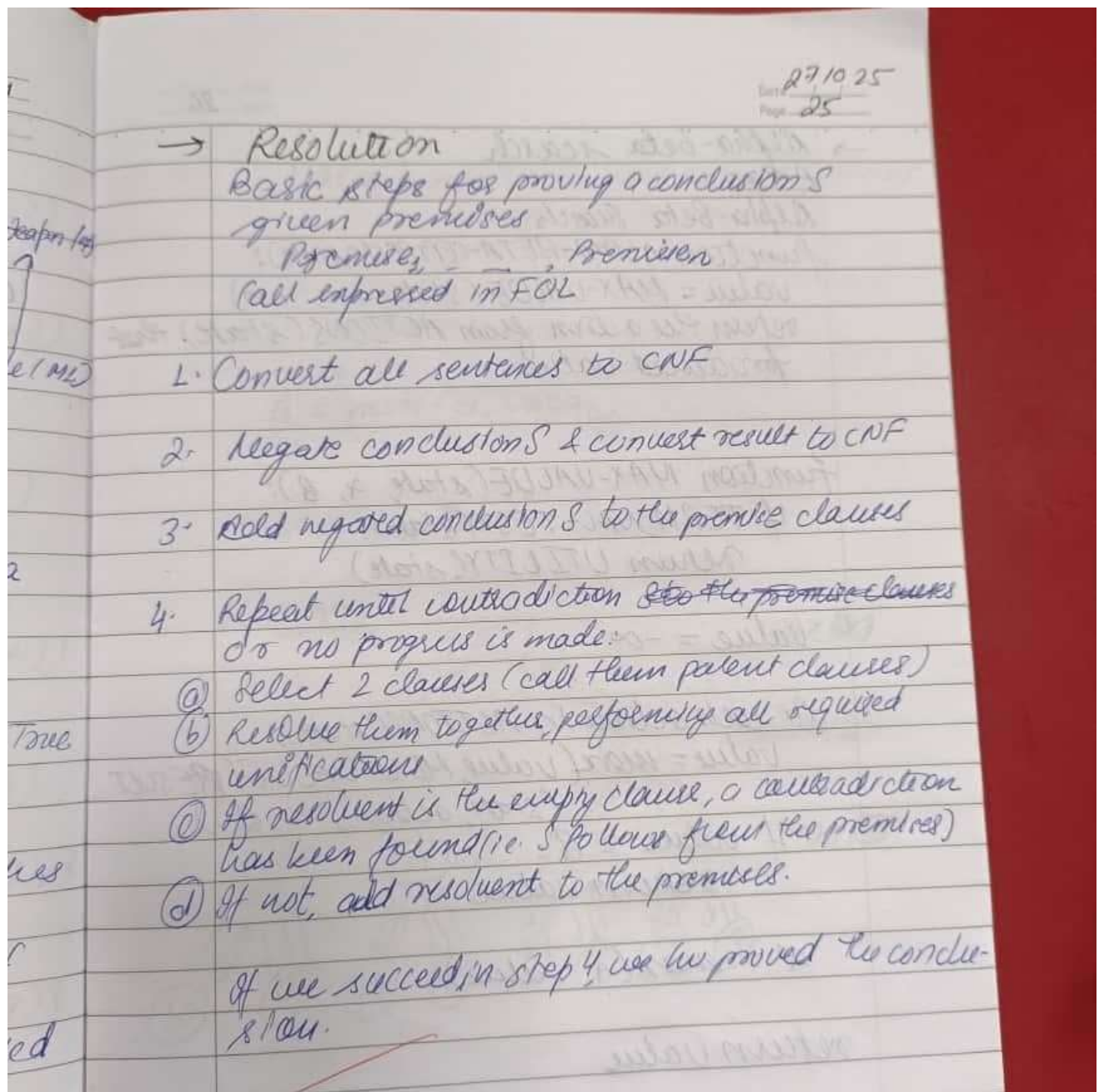
$\text{killed}(\text{Bnil}) \quad \neg \text{alive}(k) \vee \neg \text{killed}(k)$

$\neg \text{alive}(\text{Bnil}) \quad \text{alive}(\text{Bnil})$

$\{\}$

~~QED~~

29



from itertools import combinations

```
def pl_resolution(KB, query): #
    Negate the query and add to KB
    clauses = KB + [negate(query)]
    print("Initial Clauses:")
    for c in clauses:
        print(c)
    print("-" * 40)
```

```

    new = set()
while True:
    # Generate all possible pairs of clauses
    pairs = list(combinations(clauses, 2))
    for (ci, cj) in pairs:
        resolvents = resolve(ci, cj)
    if [] in resolvents:
        print(f'Derived empty clause from {ci} and {cj}')
    return True
    new.update(tuple(sorted(r)) for r in
    resolvents)

    if new.issubset(set(tuple(sorted(c)) for c in clauses)):
# No new clauses added — cannot derive contradiction
return False
    for c in new:
        if list(c) not in clauses:
            clauses.append(list(c))

```

```

def resolve(ci, cj):
    """Resolve two clauses and return the resolvents."""
    resolvents = []
    for di in ci:
        for dj in cj:
            if di == negate(dj):
                new_clause = list(set(ci + cj))
                new_clause.remove(di)
                new_clause.remove(dj)
                resolvents.append(new_clause)
    return resolvents

```

```

def negate(literal):
    """Negate a literal."""
    if literal.startswith('~'):
        return literal[1:]
    else:
        return '~' + literal

```

```

KB = [
    ['~R', 'W'],
    ['~W', 'G'],
    ['R']
]

```

```

query = 'G'

```

```

# --- Run Resolution ---
entailed = pl_resolution(KB, query)

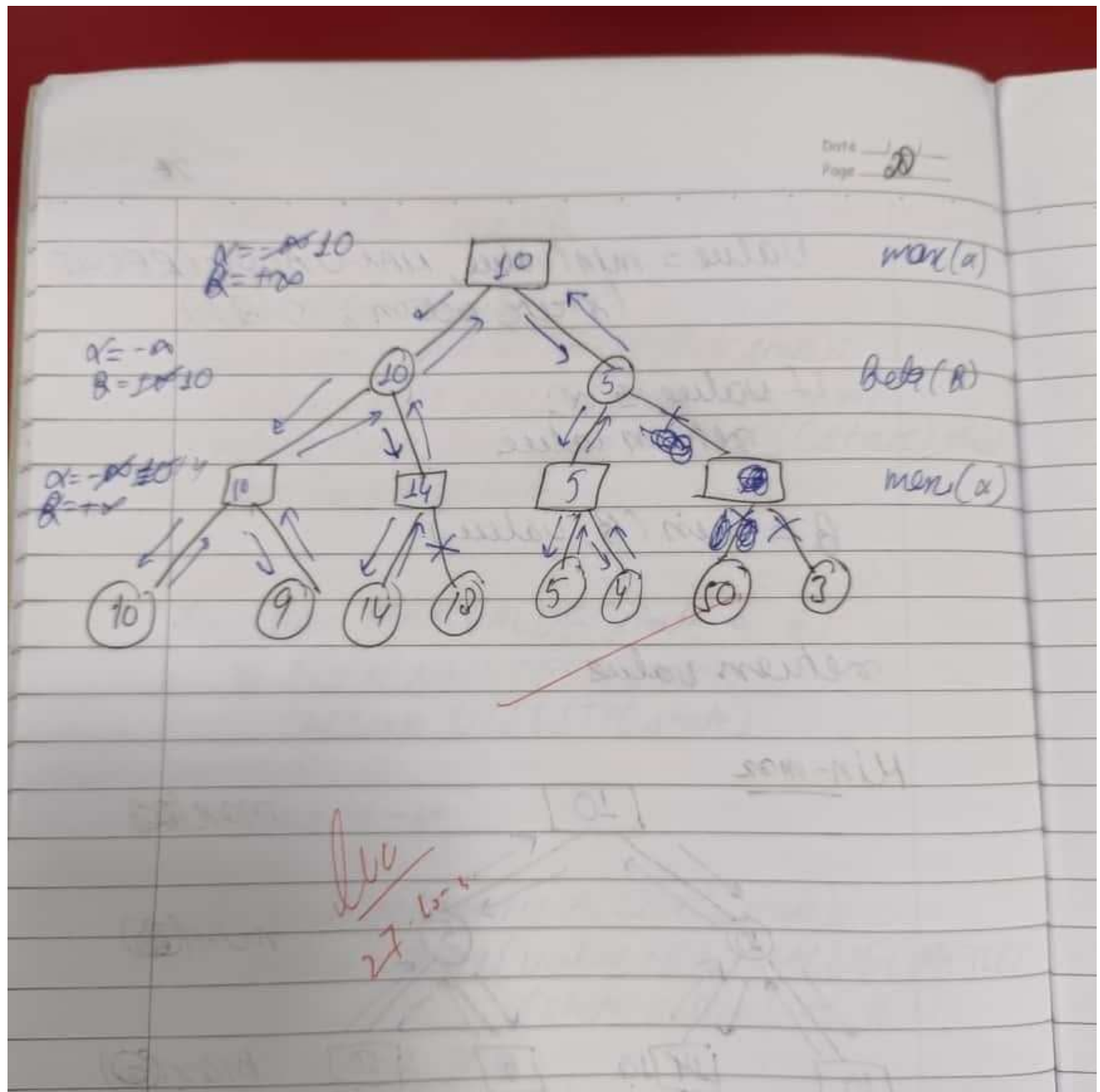
```

```
print("1BM23CS298 Sanchit
Kashyap ") print("\nResult:")
if entailed:
```

```
    print(f"✓ The knowledge base entails {query}.") else:
```

```
    print(f"✗ The knowledge base does NOT entail {query}.")
```

Program 10:
Implement Alpha-Beta Pruning



→ Alpha-beta search

Algo:-

Alpha-Beta Search

function ALPHA-BETA-CUT (state):

value = MAX-VALUE (state, $-\infty$, $+\infty$)

return the action from ACTIONS (state) that
produced value

function MAX-VALUE (state, α , β):

if TERMINAL-TEST (state):

return UTILITY (state)

value = $-\infty$

for each action in ACTIONS (state):

value = max (value, MIN-VALUE (RESULT
(state, action), α , β))

if value $\geq \beta$:

return value

α = max (α , value)

return value

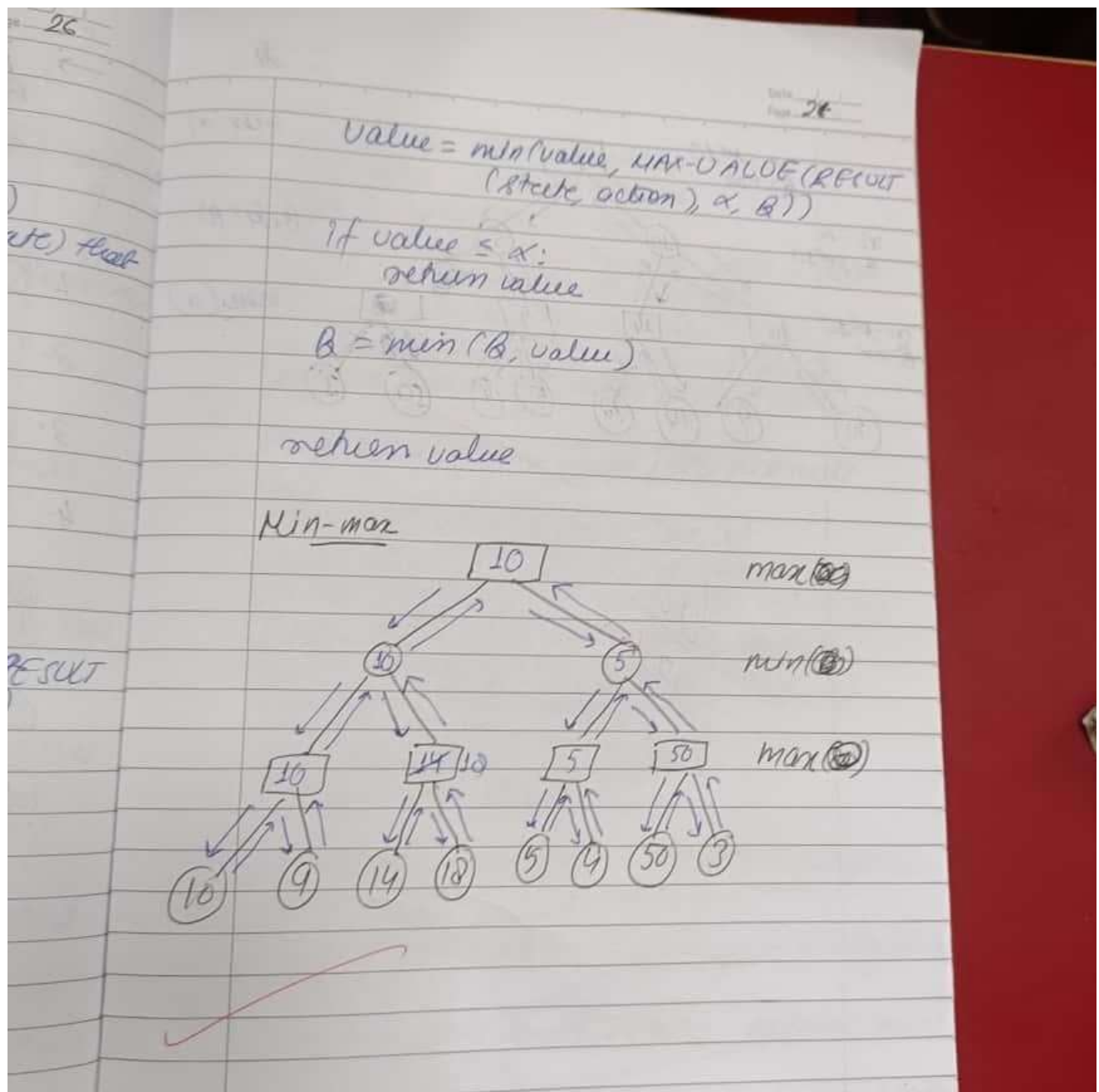
function MIN-VALUE (state, α , β)

if TERMINAL-TEST (state):

return UTILITY (state)

value = $+\infty$

for each action in ACTIONS (state):



--- Alpha-Beta Pruning Implementation ---

import math

Minimax with Alpha-Beta Pruning

```
def alphabeta(depth, node_index,
maximizing_player, values, alpha, beta, max_depth):
```

"""

depth: current depth in the game tree

```

    node_index: index of the current node in the list of values
    maximizing_player: boolean, True if it's the maximizing player's turn    values:
    list of terminal node values
    alpha, beta: alpha-beta values for pruning
    max_depth: maximum depth of the tree
    """

    # Base case: if we reach the leaf node
    if depth == max_depth:
        return values[node_index]

    if maximizing_player:
        best = -math.inf
        # Explore left and right child
        for i in range(2):
            val = alphabeta(depth + 1, node_index * 2 + i, False, values, alpha, beta, max_depth)
            best = max(best, val)
            alpha = max(alpha, best)
        # Prune
        if beta <= alpha:
            break
        return best
    else:
        best = math.inf
        # Explore left and right child
        for i in range(2):
            val = alphabeta(depth + 1, node_index * 2 + i, True, values, alpha, beta, max_depth)
            best = min(best, val)
            beta = min(beta, best)
        # Prune
        if beta <= alpha:
            break
        return best

# --- Example Game Tree ---

# Terminal node values (leaf nodes)
values = [3, 5, 6, 9]

# Tree depth = log2(len(values)) = 2
max_depth = 2

# Run alpha-beta pruning
best_value = alphabeta(0, 0, True, values, -math.inf, math.inf, max_depth)

print("1BM23CS298 Sanchit Kashyap ")
print(f"The optimal value is: {best_value}")

```